# Lecture 6 — Two Pointers & Sliding Window

> Year Curriculum · Prerequisites: Weeks 3–5 (complexity and amortized analysis, sorting, binary search) · Estimated time: 2 hours lecture + 4 hours practice

## 1. Overview

Many problems ask about **pairs** of elements or about **contiguous subarrays**. The brute-force approach tries every pair or every (l, r) and takes $O(n^2)$ time, which is too slow for n = 2·10^5. The **two-pointer technique** does the same job in O(n). It keeps two indices that only ever move forward, so the total number of moves is at most 2n.

The technique works whenever the problem has the right kind of **monotonicity**: moving one pointer in one direction can only make a condition "more true" or "less true", never both. This lecture covers:

1. **Opposite-end pointers** on a sorted array: pair sums and k-sum.
2. **Same-direction pointers (sliding windows)** with variable size: longest or shortest valid subarray, and counting valid subarrays.
3. Windows maintained with a frequency table (distinct values, at most k distinct).
4. Fixed-size windows and merging sorted sequences.
5. Recognizing when the technique **does not** apply, typically because negative numbers break monotonicity, and what to do instead (Week 7).

## 2. Opposite-end pointers

### 2.1 Pair with a given sum

**Problem (CSES 1640 — Sum of Two Values).** Given an array and a target x, find two positions i ≠ j with a_i + a_j = x, or report IMPOSSIBLE.

Sort the array, keeping the original indices because the output needs them. Place `i` at the start and `j` at the end:

- If a_i + a_j = x, we are done.
- If a_i + a_j < x, then a_i is too small to pair with **any** remaining element, since a_j is the largest remaining. Discard it: `i++`.
- If a_i + a_j > x, then a_j is too large to pair with any remaining element, since a_i is the smallest. Discard it: `j--`.

Each step discards one element, so there are at most n − 1 steps. With the sort, the total is O(n log n).

**Correctness.** Suppose a valid pair (i*, j*) exists with i* < j*. Elements are discarded only when they cannot be part of *any* valid pair with the remaining elements, as argued above. A valid pair is therefore never destroyed while both of its elements remain. Formally: if `i` reaches i* while `j` > j*, then a_{i*} + a_j ≥ a_{i*} + a_{j*} = x. Either the sum equals x and we stop with a valid pair, or it exceeds x and `j` moves toward j*. The symmetric argument holds when `j` reaches j* first.

**Worked example.** a = [2, 7, 5, 1], x = 8. Sorted with indices: (1,4) (2,1) (5,3) (7,2).

- i → 1, j → 7: sum 8 = x. Answer: positions 4 and 2.

Week 2 solved the same problem with a hash map in expected O(n). Both approaches are worth knowing.

### 2.2 Three values and beyond

**CSES 1641 — Sum of Three Values** (n ≤ 5000): fix the first element i, then run the two-pointer scan on the suffix (i+1 … n−1) with target x − a_i. That gives O(n²) total, about 2.5·10^7 steps, compared with O(n³) ≈ 1.25·10^11 for brute force. In general, k-sum takes O(n^(k−1)) this way.

### 2.3 Other opposite-end patterns

- **Ferris Wheel (CSES 1090, Week 4)**: pair the heaviest with the lightest.
- **Palindrome checks**: compare s[i] with s[j] while moving inward.
- **Container with most water**: always move the pointer at the shorter side.

## 3. Sliding windows (same-direction pointers)

### 3.1 The precondition

We consider windows [l, r] and some property `valid(l, r)`. The technique needs:

> If [l, r] is valid, then every sub-window [l', r] with l' ≥ l is also valid.

Shrinking from the left preserves validity. Equivalently, for each r there is a smallest valid left end `best_l(r)`, and **best_l(r) never decreases as r increases**. Examples:

- "sum ≤ t" with non-negative elements: removing elements cannot increase the sum. ✓
- "all elements distinct": removing elements cannot create duplicates. ✓
- "at most k distinct values". ✓

- "sum = x" with **negative** elements allowed: ✗. Removing a negative element increases the sum.

### 3.2 The template

```
int l = 0;
for (int r = 0; r < n; r++) {
    add(a[r]);
    while (!valid()) remove(a[l++]);
    // now [l, r] is the longest valid window ending at r
    best = max(best, r - l + 1);
}
```

**Complexity (amortized).** The outer loop runs n times. The inner `while` loop can run many times for a single r, but `l` only increases and never exceeds n, so across the *whole* run the inner body executes at most n times. With O(1) `add`/`remove`, the total is O(n). With a `map` for frequencies, it is O(n log n).

### 3.3 Longest valid window — CF 279B Books

Reading times a_i ≥ 1 and total free time t. Find the maximum number of consecutive books that can be read. A window is valid if its sum is ≤ t. Apply the template with a running sum.

Trace with a = [3, 1, 2, 1], t = 5:

| r | add | sum | shrink | window | best |
|---|---|---|---|---|---|
| 0 | 3 | 3 | — | [3] | 1 |
| 1 | 1 | 4 | — | [3,1] | 2 |
| 2 | 2 | 6 | drop 3 → 3 | [1,2] | 2 |
| 3 | 1 | 4 | — | [1,2,1] | 3 |

### 3.4 Shortest window satisfying a "large enough" property

For properties where *expanding* keeps validity, such as "sum ≥ s" with non-negative elements, find the shortest valid window: after adding a[r], shrink **while the window stays valid** after removing a[l], and record r − l + 1 whenever the window is valid. **CF 701C — They Are Everywhere** is this pattern: the shortest window containing every distinct letter of the string.

### 3.5 Counting subarrays

The number of valid windows ending at r is exactly **r − l + 1** (all left ends from l to r), because validity is preserved by shrinking. Summing over r counts all valid subarrays in O(n):

- **Subarrays with sum ≤ x** (non-negative elements): `ans += r - l + 1`.
- **CSES 2428 — Subarray Distinct Values**: subarrays with at most k distinct values. Keep a frequency map and a `distinct` counter. Shrink while distinct > k, then add r − l + 1.
- **Exactly k**: count(at most k) − count(at most k − 1). This works because "at most k" is monotone, while "exactly k" is not.
- **CSES 1660 — Subarray Sums I**: positive elements, count subarrays with sum **exactly** x. Shrink while the sum is > x. Then, for fixed r, the sums over left ends are strictly decreasing as l increases, so at most one left end has sum exactly x, namely the current l if sum == x.

The answer can be as large as n(n+1)/2 ≈ 2·10^10 for n = 2·10^5. **Use `long long` for counts.**

### 3.6 Windows with a frequency table — CSES 1141 Playlist

Find the longest segment of consecutive songs with no repeated song. Track how many times each value appears in the window. After adding k[r], shrink while k[r] appears more than once, since only the newly added value can be duplicated. Song IDs go up to 10^9, so use a `map` (O(n log n)), or coordinate-compress first and use a plain array (O(n)).

An alternative formulation stores, for each value, its last position: `l = max(l, last[k[r]] + 1)`. This is the same idea with a jump instead of a loop.

### 3.7 Fixed-size windows

When the window length k is fixed, slide it by adding a[i] and removing a[i − k]. Sums, counts of a value, and parity work this way. **Maximum or minimum** over each window cannot be maintained by simple add and remove, because when the maximum leaves you do not know the next one. It needs a monotonic deque (Week 12) or a multiset (O(log n) per step).

## 4. Pointers over two sequences

**Merging.** Two sorted arrays can be merged in O(n + m) by repeatedly taking the smaller front element. This is the merge step of merge sort.

**CSES 1084 — Apartments.** Applicants want sizes in [a_i − k, a_i + k], and there are apartments of sizes b_j. Maximize the number of matches. Sort both lists and walk them together. If apartment j is too small for applicant i, skip the apartment (`j++`). If it is too large, skip the applicant (`i++`). Otherwise match both. Each step advances a pointer, so the walk is O(n + m) after sorting. Greedy correctness follows from an exchange argument as in Week 4: matching the smallest applicant with the smallest fitting apartment never hurts.

**Counting pairs across arrays.** For example, count pairs (i, j) with a_i < b_j. With both arrays sorted, as `i` increases the boundary in b moves only right.

## 5. When two pointers fails

Consider "count subarrays with sum exactly x" when elements can be negative. With a = [2, −1, 2] and x = 3, the whole array works but the window logic breaks: adding −1 *decreases* the sum, so "too big → shrink" is no longer sound. The correct tool is **prefix sums with a hash map** (Week 7, CSES 1661 Subarray Sums II), which counts in O(n) without any monotonicity.

The same applies to any property where shrinking can turn a valid window invalid. Before writing two pointers, state the precondition in Section 3.1 and convince yourself it holds.

**Two pointers or binary search?** Anything solvable with a sliding window can usually also be solved by binary searching r for each l, using prefix sums to evaluate a window in O(1). That costs O(n log n). Two pointers are faster and shorter, but binary search can be easier to get right under pressure.

## 6. Reference implementations

### 6.1 Sum of Two Values (CSES 1640)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long x;
    cin >> n >> x;
    vector<pair<long long, int>> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i].first;
        a[i].second = i + 1;
    }
    sort(a.begin(), a.end());
    int i = 0, j = n - 1;
    while (i < j) {
        long long s = a[i].first + a[j].first;
        if (s == x) {
            cout << a[i].second << ' ' << a[j].second << '\n';
            return 0;
        }
        if (s < x) i++;
        else j--;
    }
    cout << "IMPOSSIBLE\n";
    return 0;
}
```

### 6.2 Subarray Distinct Values (CSES 2428), counting windows with ≤ k distinct values

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<int> a(n);
    for (auto &v : a) cin >> v;
    map<int, int> cnt;
    int distinct = 0, l = 0;
    long long ans = 0;
    for (int r = 0; r < n; r++) {
        if (cnt[a[r]]++ == 0) distinct++;
        while (distinct > k) {
            if (--cnt[a[l]] == 0) distinct--;
            l++;
        }
        ans += r - l + 1;
    }
    cout << ans << '\n';
    return 0;
}
```

Sample: `5 2` / `1 2 3 1 1` → `10`.

### 6.3 Playlist (CSES 1141), longest window with distinct values

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> k(n);
    for (auto &v : k) cin >> v;
    map<int, int> last;                // value -> last index seen
    int l = 0, best = 0;
    for (int r = 0; r < n; r++) {
        auto it = last.find(k[r]);
        if (it != last.end()) l = max(l, it->second + 1);
        last[k[r]] = r;
        best = max(best, r - l + 1);
    }
    cout << best << '\n';
```

```
    return 0;
}
```

Sample: `8` / `1 2 1 3 2 7 4 2` → `5` (the segment 1 3 2 7 4).

### 6.4 Subarray Sums I (CSES 1660)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long x;
    cin >> n >> x;
    vector<long long> a(n);
    for (auto &v : a) cin >> v;
    long long sum = 0, count = 0;
    int l = 0;
    for (int r = 0; r < n; r++) {
        sum += a[r];
        while (sum > x) sum -= a[l++];
        if (sum == x) count++;
    }
    cout << count << '\n';
    return 0;
}
```

Sample: `5 7` / `2 4 1 2 7` → `3`.

## 7. Common mistakes

1. **No monotonicity.** Using a sliding window with negative numbers, or with a property that shrinking can break. Write the precondition down.
2. `int` **overflow in counts.** Summing r − l + 1 over all r reaches about $n^2/2$.
3. **Incrementing** `l` **before removing a[l]**, or the reverse in the wrong place. `remove(a[l++])` does it in the right order.
4. **Empty windows.** In the Books problem, if a single element exceeds t, the window becomes empty (l = r + 1) and r − l + 1 = 0. That is correct, but conditions such as `while (l < r && ...)` silently prevent it.
5. **Losing original indices when sorting** for pair-sum outputs.
6. **Two pointers on unsorted data** in opposite-end problems.
7. **Using i == j** in pair problems where i ≠ j is required. The `i < j` loop condition handles it.
8. **Shortest versus longest windows.** For "longest", shrink until valid. For "shortest", shrink while still valid. Mixing them up gives wrong answers on the samples, so trace a sample by

hand.

## 8. Practice problems

### CSES

- 1640 — Sum of Two Values — https://cses.fi/problemset/task/1640
- 1641 — Sum of Three Values — https://cses.fi/problemset/task/1641
- 1660 — Subarray Sums I — https://cses.fi/problemset/task/1660
- 1141 — Playlist — https://cses.fi/problemset/task/1141
- 2428 — Subarray Distinct Values — https://cses.fi/problemset/task/2428
- 1084 — Apartments — https://cses.fi/problemset/task/1084

### Codeforces

- 279B — Books — https://codeforces.com/problemset/problem/279/B
- 6C — Alice, Bob and Chocolate — https://codeforces.com/problemset/problem/6/C
- 1354B — Ternary String — https://codeforces.com/problemset/problem/1354/B
- 701C — They Are Everywhere — https://codeforces.com/problemset/problem/701/C
- 660C — Hard Process — https://codeforces.com/problemset/problem/660/C

### AtCoder

- ABC098 D — Xor Sum 2 — https://atcoder.jp/contests/abc098/tasks/arc098_b (a window is valid while XOR equals sum, which is monotone because bits must not overlap)

### Course

- Codeforces EDU, "Two Pointers Method" (steps 1–3): https://codeforces.com/edu/courses

## 9. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Section 8.1 ("Two pointers method"): https://cses.fi/book/book.pdf
- USACO Guide, Silver module "Two Pointers": https://usaco.guide/silver/two-pointers
- Steven Halim and Felix Halim, *Competitive Programming 4*, sections on sliding windows
- Week 7 (prefix sums), for subarray problems without monotonicity, and Week 12 (monotonic deque), for sliding-window min/max